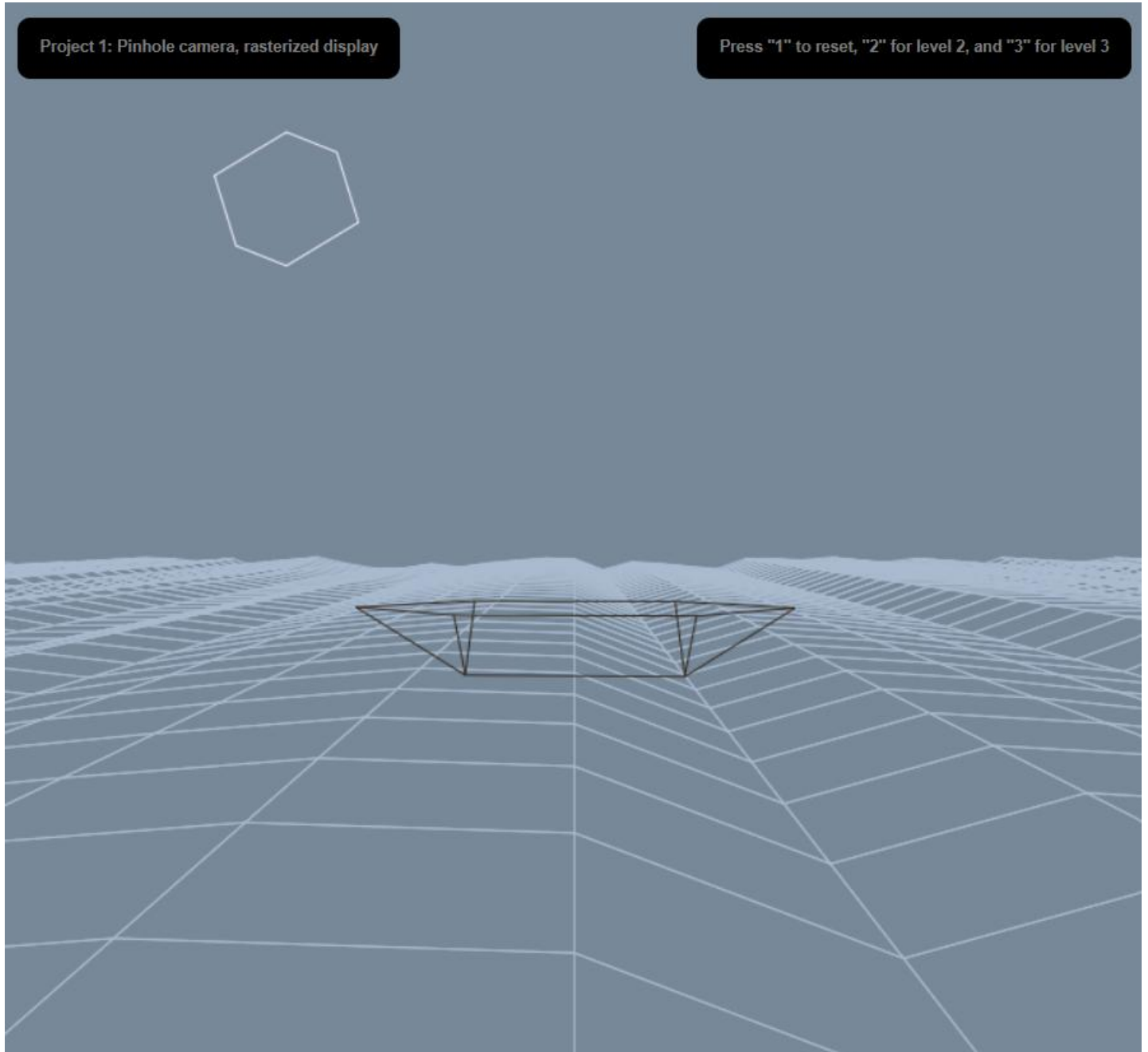


Project 1: Pinhole Camera, Rasterized Display Documentation

Grant Hamilton



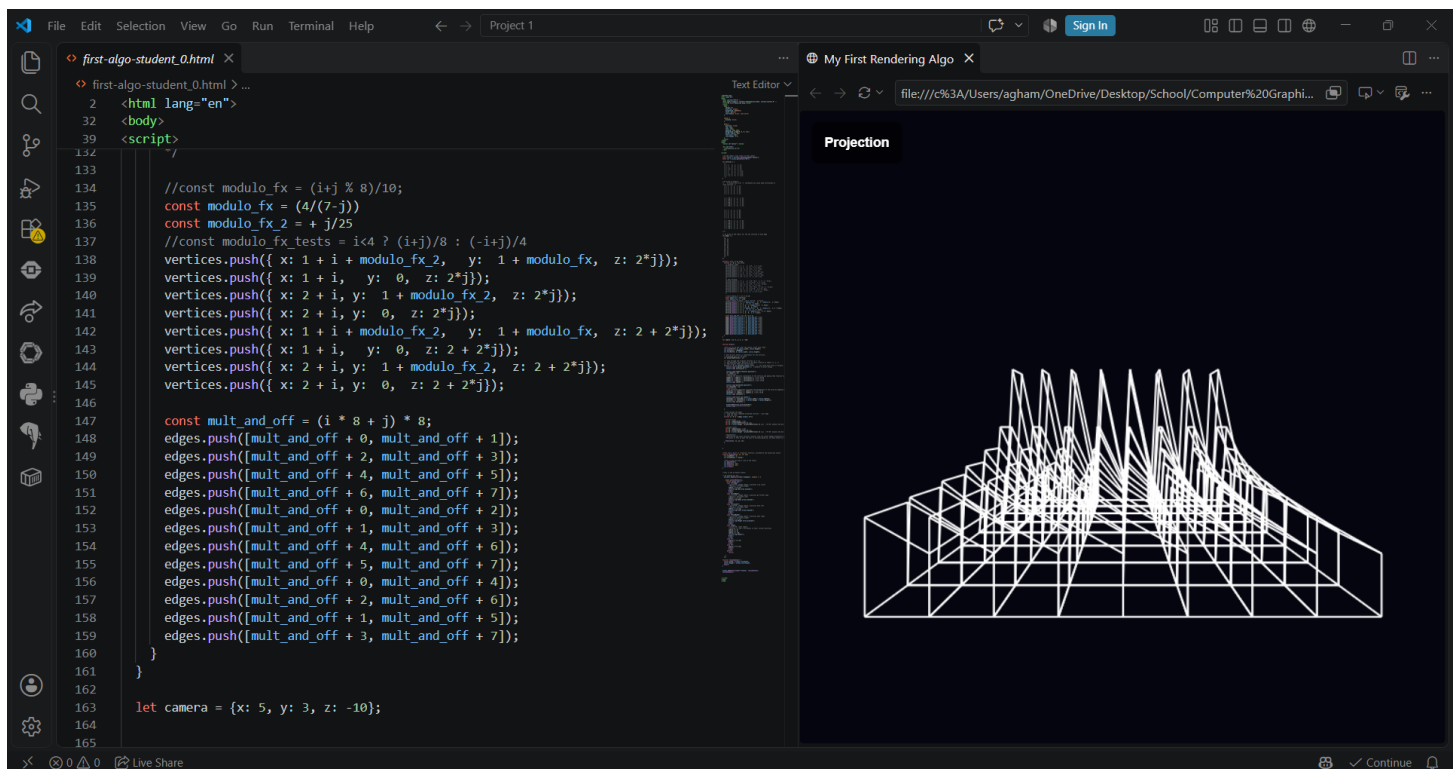
Design Plans, Features and Controls

When I started this project, I wasn't sure what direction to take it. Early on, I wanted to plan this like an art project. I was inspired by a YouTube video where the speaker talked about this strobing effect he'd experienced when driving along a field. The crop was arranged in a grid-like fashion, and it created an interesting visual:

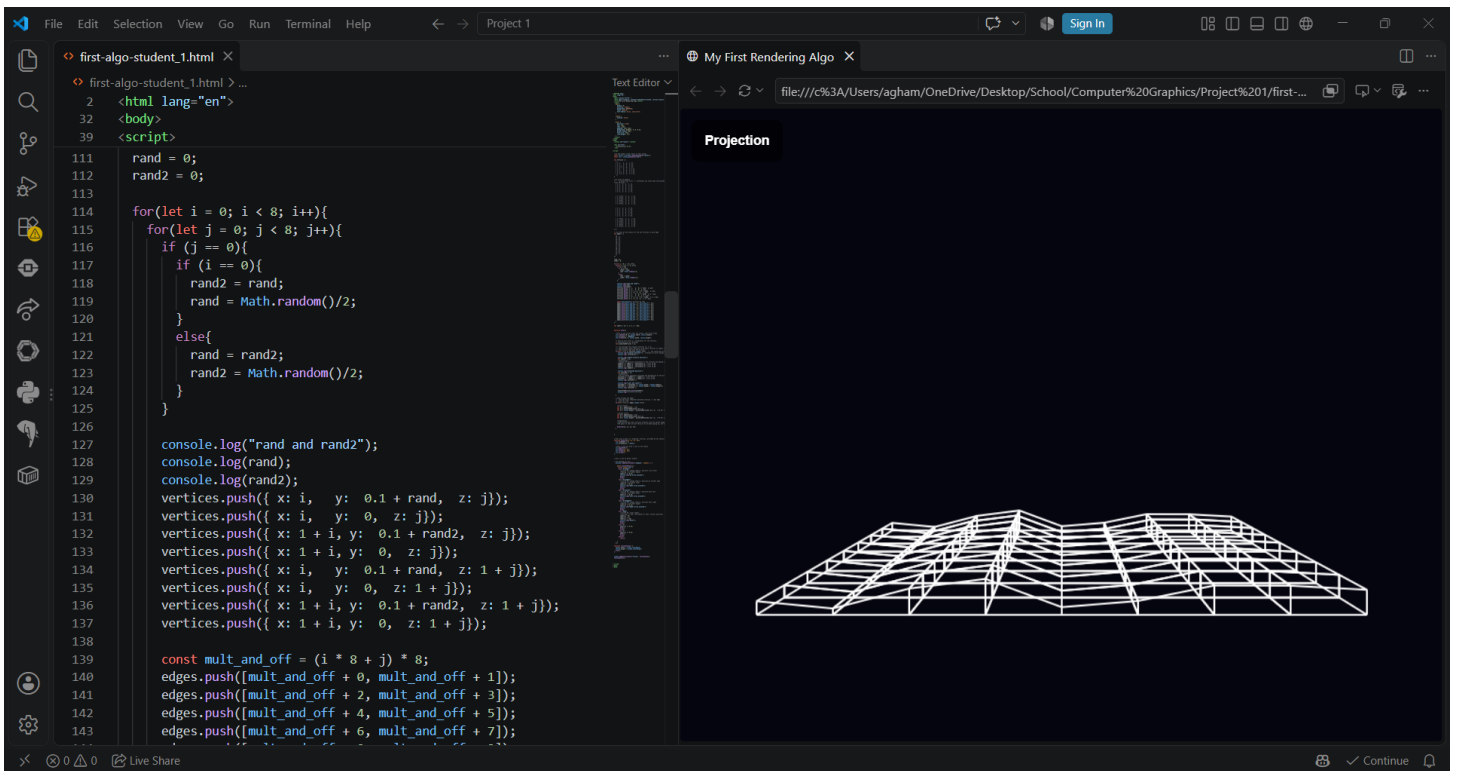


[Video](#) by Tor Parsons with the strobing effect

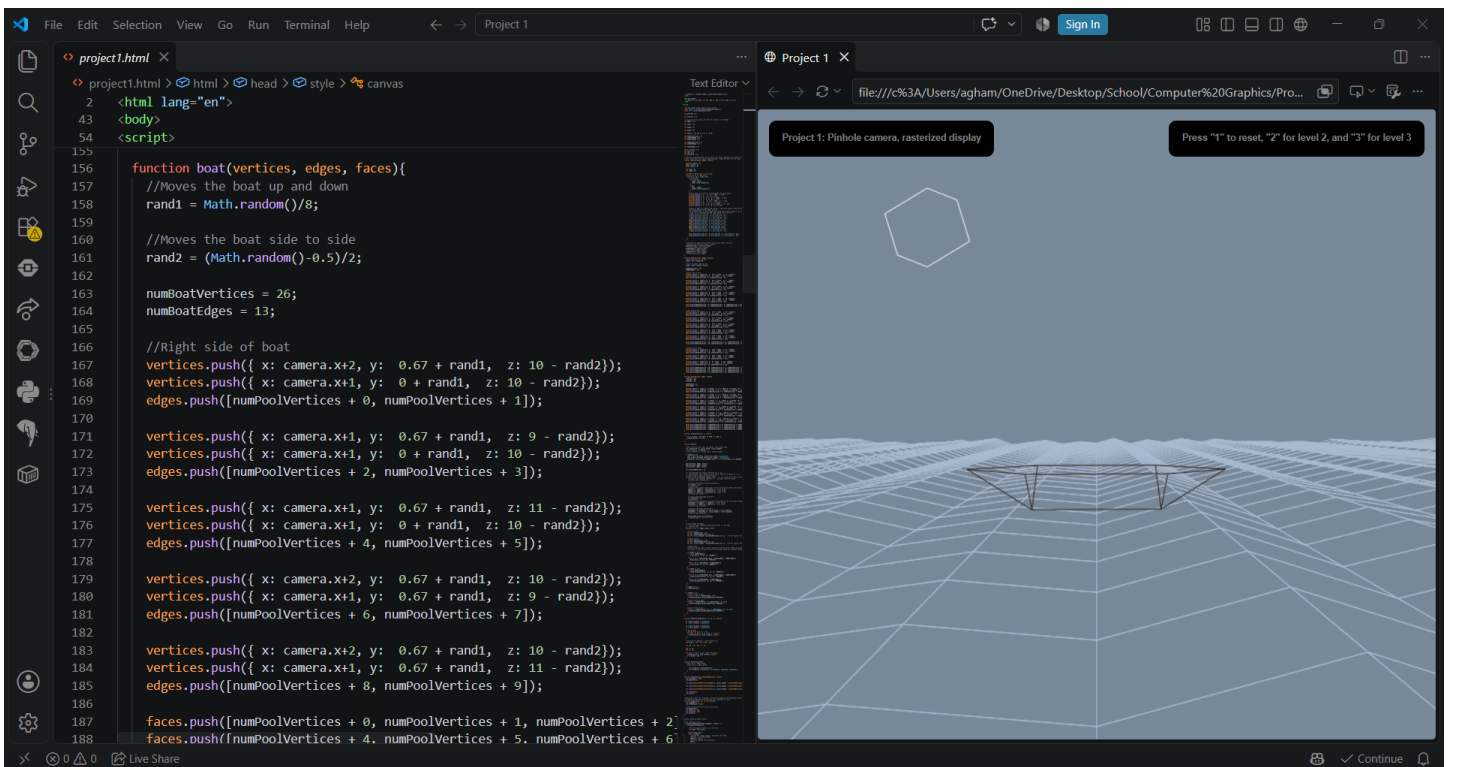
I wanted to explore the idea of creating complex patterns with simple geometry. I started with a grid of lines, like the image above. After some tinkering, these lines became cubes placed next to each other. Because these cubes were generated by a nested for loop, I made the vertices' locations dependent on the iteration.



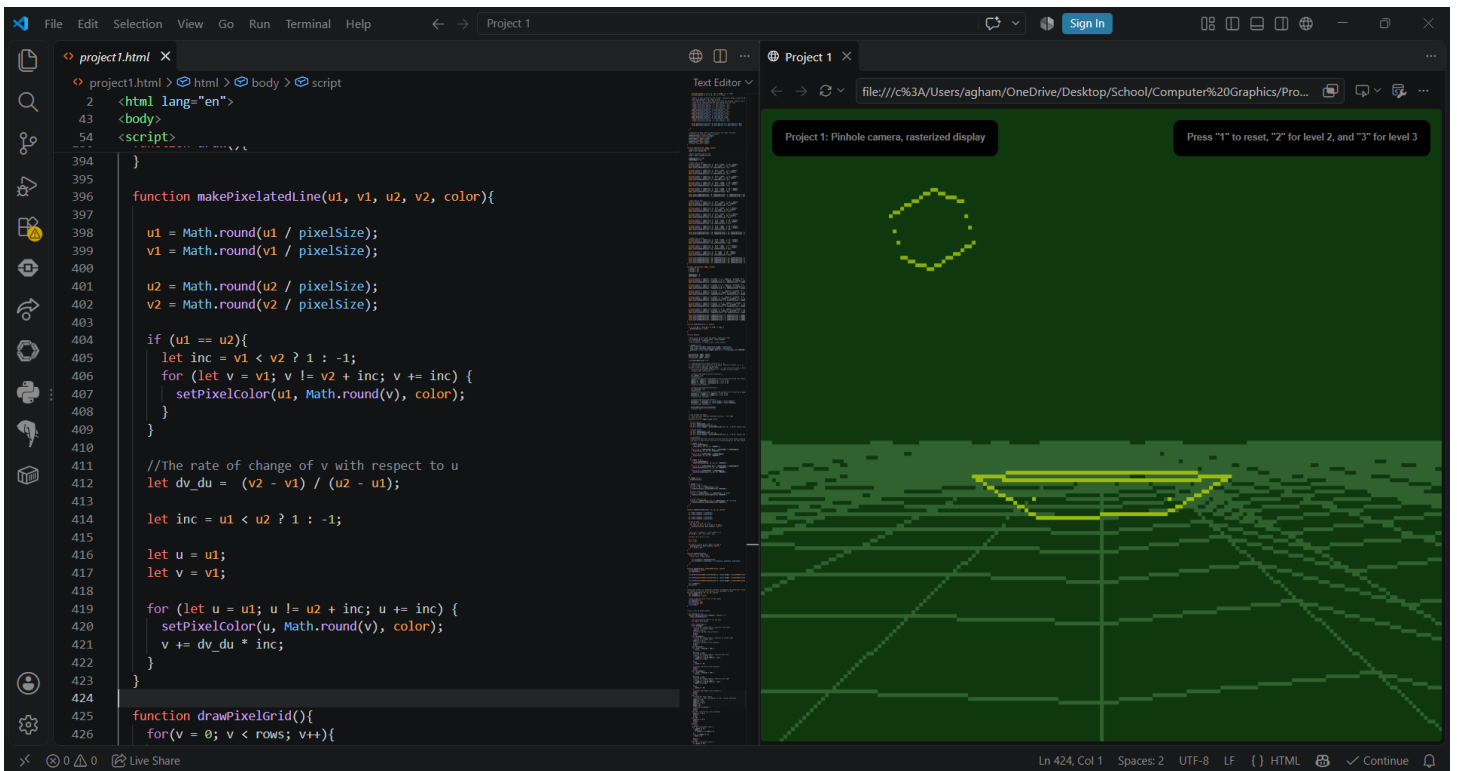
I messed around with the for loops a lot without giving it much thought. I couldn't really get at what I wanted, so I kept tinkering.



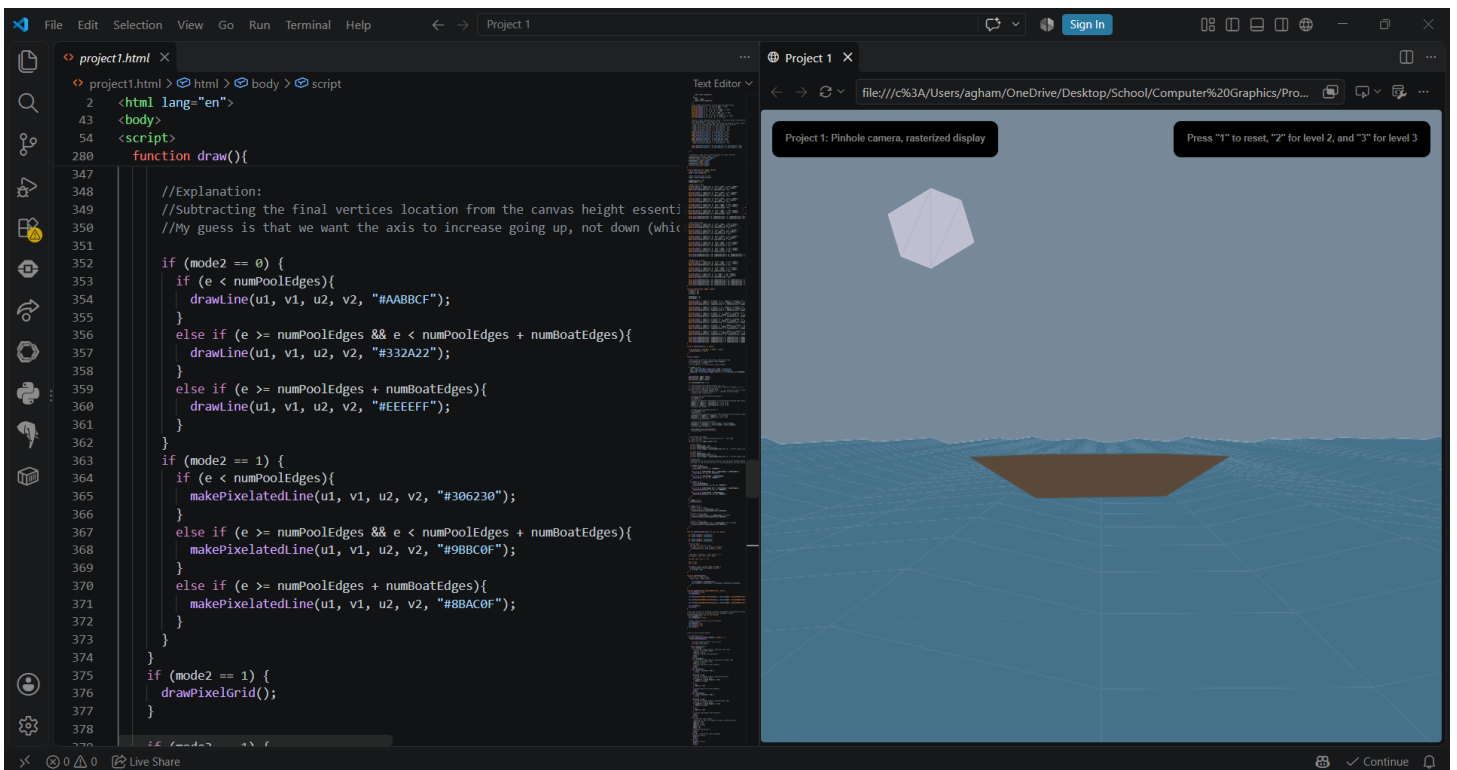
Eventually, I output this image above. By now I'd started using `Math.random()` to generate different heights for each mesh. I wanted to create some uniformity, so I gave a row of vertices the same height. Right off the bat it reminded me of a wave of water. Refreshing the page would randomize the heights again, so I ended up going the wave route. Due to how slow everything was running, I kept my grid of meshes to 8x8, but once I realized it was faster without console logging enabled, I expanded to 20x20.



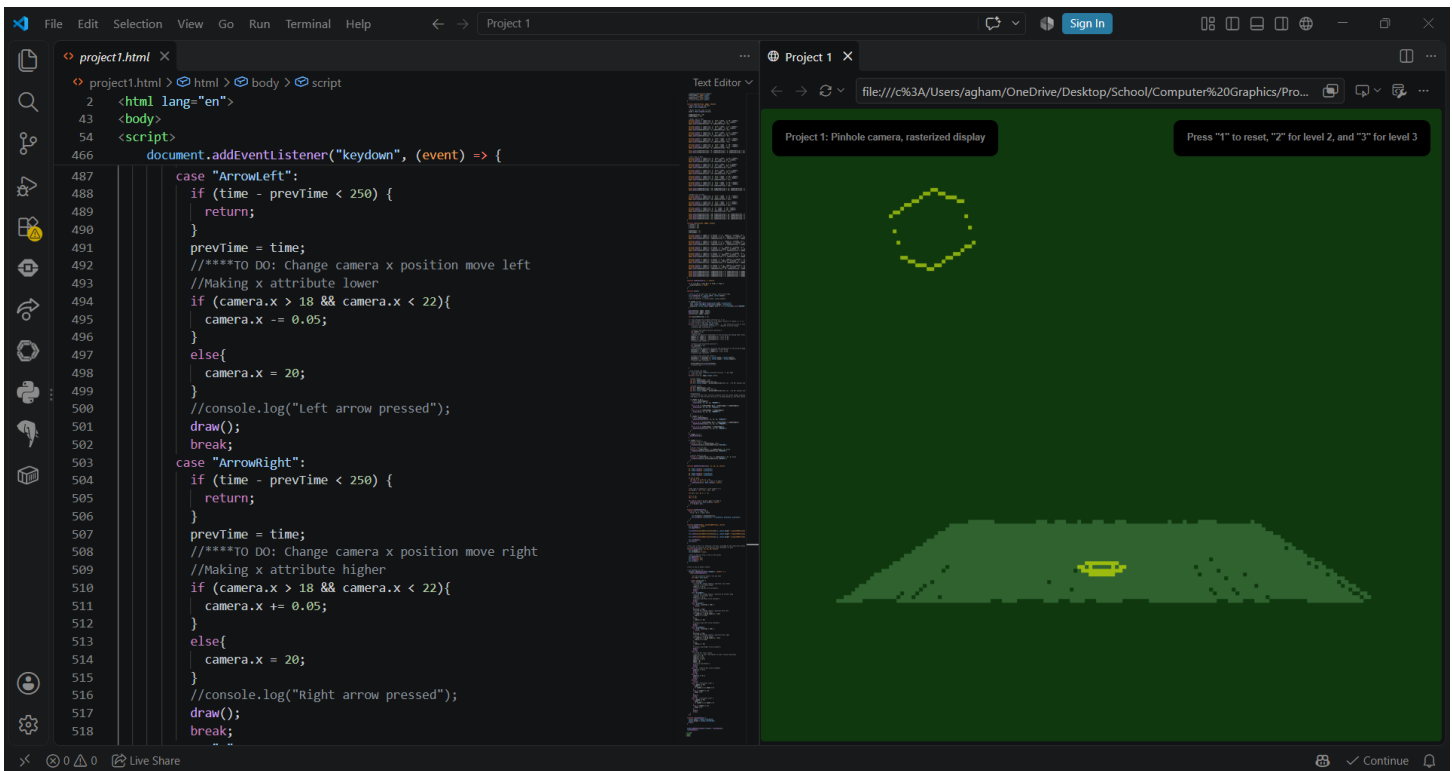
Lastly, I created a boat and a sun to wrap up the environment. I liked how the random functionality of the water looked, so I applied some movement to the boat as well to simulate it rocking along the waves.



I'm a big fan of pixel art and the techniques that video game developers used on retro systems. While it might not be realistic to have a 3D scene like this on a Gameboy, I thought its shades of green complemented the design.



For level 3, I kept the colors of the faces desaturated. I figured a grim, rainy day would make sense with the movement of the waves. Unfortunately, I realized a bit too late that level 3 needed to be pixelated like level 2, so that's why it's in full resolution.



I wasn't happy with how quickly the waves would move when traversing the environment, so I restricted the amount of updates that can occur when moving left and right. Because the boat is locked to the camera's x axis, I wanted to make the movement of the boat along with waves as realistic as possible.

There's the option to move up, down, forward, and backward without any speed restriction. I consider those as secondary options to tinker with the vantage point. I zoomed out in the image above so you can see the grid. I didn't want the user to move too far left or right, so once they get to a certain point it resets back home.

All controls are as follows:

"1" – Resets the camera, along with the environment to level 1 of the project

"2" – Toggles to level 2 of the project

"3" – Toggles to level 3 of the project

"4" – Moves the camera down

"5" – Moves the camera up

Up, down, left, and right keys move forward, backward, left and right respectively.

Implementation

`pool(vertices, edges, faces)`, `boat(vertices, edges, faces)` and `sun(vertices, edges, faces)` generate the vertices, edges, and faces of their respective objects. The first two generate these through nested for loops, whereas the sun's makeup is completely static. The number of vertices, edges, and faces are also recorded here to get the coloring correct later down the line.

`setPixelColor(u, v, color)` assigns a color to a pixel for level 2. I added a conditional for checking bounds, but otherwise it's like the example function from the professor.

`draw()` is what does the brunt of the work when it comes to updating the screen. It calls `pool()`, `boat()` and `sun()` to get its vertices, edges, and faces, then projects them to the screen through acting as a pinhole camera. It accounts for the camera's relative coordinates, along with dividing the coordinates of the objects to get the right perspective. This function also chooses between the method of line drawing by determining the mode, and houses all the color data for the edges and faces.

`makePixelatedLine(u1, v1, u2, v2, color)` is a function for level 2 that focuses on drawing the lines in a pixelated fashion. It calculates the slope the edge makes with respect to the camera, thus why it calls for `u1`, `v1`, `u2`, and `v2`. The implementation originally struggled with vertical lines and lines moving right and down, so the algorithm has some handling made for those instances.

`drawPixelGrid()` is a simple helper for initializing the level 2 grid.

`drawFace(face, projectedVertices, color)` is the function for rendering triangles. It makes heavy use of the canvas library by taking the camera's perspective of each face's vertices and connects them together. There's also an argument for color to make sure the objects each pop out.

`drawLine(x1, y1, x2, y2, color)` also makes use of JavaScript's canvas library to fill in lines. It acts in a similar way to `drawFace` as it's finding the relative position of the vertices to the camera, then connecting them, only this time by a single line. This function also has a color argument.

`addEventListener(...)` is a function for checking keyboard inputs.

Future Work

I was pleasantly surprised by how fluid the waves and the boat were able to look with such simple animation. Taking away this aspect of the project, there isn't much going on to engage the viewer. I think it could benefit from more objects – like a beach in the foreground, or a low-poly fisherman on the boat. I also think the sky is lacking a lot of detail, so maybe some clouds could be added too.



[Image](#) by Alex Grichenko

Along with that, I could make the algorithm for the waves more complex. Right now, they span up and down the grid, but it might not be that difficult to build on that.



[Image](#)